

```

"@nestjs/bullmq":
version: "^11.0.5"
context7_id: "/nestjs/docs.nestjs.com"
official_docs: "https://docs.nestjs.com/techniques/queues"
fetch_at: "2026-08-24T18:56:00-03:00"
bullmq:
version: "^5.81.3"
context7_id: "/taskforcesh/bullmq"
official_docs: "https://docs.bullmq.io/"
fetch_at: "2026-08-24T18:56:00-03:00"
ioredis:
version: "^5.11.1"
context7_id: "/redis/ioredis"
official_docs: "https://github.com/redis/ioredis#readme"
fetch_at: "2026-08-24T18:56:00-03:00"
"@aws-sdk/client-s3":
version: "^3.x"
context7_id: "/aws/aws-sdk-js-v3"
official_docs: "https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/client/s3/"
fetch_at: "2026-08-24T18:56:00-03:00"
"@aws-sdk/s3-request-presigner":
version: "^3.x"
context7_id: "/aws/aws-sdk-js-v3"
official_docs: "https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/Package/-aws-sdk-s3-request-presigner/"
fetch_at: "2026-08-24T18:56:00-03:00"
nanoid:
version: "^3.3.18"
context7_id: "/ai/nanoid"
official_docs: "https://github.com/ai/nanoid#readme"
fetch_at: "2026-08-24T18:56:00-03:00"
sources_mtime:
docs/decisions/technical-decisions-phase-03-videos.md: "2026-08-24T18:54:35-03:00"

```

phase-03-videos — Library References

Distilled docs for libraries decided in this phase (TD-01, TD-02, TD-03, TD-06, TD-07). Re-fetch when the underlying TD changes.

TD-05 note (no npm dependency): FFmpeg/ffprobe are invoked via Node's built-in `child_process (execFile)`. `fluent-ffmpeg / @types/fluent-ffmpeg` are **deliberately not installed** — the package was archived/unmaintained on 2025-05-22 (TD-05 Option A rejected). The `ffmpeg` and `ffprobe` **binaries** are installed in the worker Docker image (`apt-get install -y ffmpeg`), not as npm packages.

Install command

Production dependencies (run inside `nestjs-project/`):

```
npm install @nestjs/bullmq@^11.0.5 bullmq@^5.81.3 ioredis@5.11.1 @aws-sdk/client-s3@^3 @aws-sdk/s3-request-presigner@^3 nanoid@^3.3.18
```

Version-pin rationale:

- `bullmq@5.x` — the mature stable line; `@nestjs/bullmq@11` peer range is `^3 || ^4 || ^5 || ^6`, so 5.x is supported. (v6 also works; 5.x chosen for stability.)
- `ioredis@5.x` — `bullmq@5` depends on `ioredis@5.11.1`; installing it explicitly documents the connection dependency (used to build the shared `connection` object).
- `nanoid@3.x` — the **CommonJS-compatible** line. `nanoid 4+/5+/6+` are ESM-only and break the NestJS CommonJS build without dynamic import (TD-06).
- AWS SDK v3 packages are versioned in lock-step; `^3` pulls the current 3.x client + presigner together.

No new **devDependencies** are required (`@types/*` for the AWS SDK and `nanoid` ship their own types; `bullmq` and `ioredis` bundle types).

@nestjs/bullmq (TD-01, TD-04)

Source: official NestJS Queues docs — <https://docs.nestjs.com/techniques/queues>. Maps to `phase-03-videos/TD-01` (queue) and `phase-03-videos/TD-04` (separate worker container).

Module registration (shared connection)

```
// src/queue/queue.module.ts
import { BullModule } from '@nestjs/bullmq';

BullModule.forRootAsync({
  inject: [queueConfig.KEY],
  useFactory: (cfg: ConfigType<typeof queueConfig>) => ({
    connection: { host: cfg.redisHost, port: cfg.redisPort },
  }),
});

// register the named queue everywhere it is produced/consumed
BullModule.registerQueue({ name: 'video-processing' });
```

Producer (API side)

```
import { InjectQueue } from '@nestjs/bullmq';
import { Queue } from 'bullmq';

constructor(@InjectQueue('video-processing') private readonly queue: Queue) {}

await this.queue.add(
  'process-video', // job name
  { videoId, storageKey } as VideoJobData,
  {
    attempts: 3, // TD-08 retry policy
    backoff: { type: 'exponential', delay: 5000 },
    removeOnComplete: true,
    removeOnFail: false, // keep failed jobs = dead-letter set
  },
);
```

Consumer (worker side)

```
import { Processor, WorkerHost } from '@nestjs/bullmq';
import { Job } from 'bullmq';

@Processor('video-processing')
export class VideoProcessor extends WorkerHost {
  async process(job: Job<VideoJobData>): Promise<void> {
    // ffprobe metadata + ffmpeg thumbnail, then update status → ready
  }
}
```

- `WorkerHost.process()` is the canonical `@nestjs/bullmq v11` pattern (replaces the old `@Process()` method decorator).
- A job that throws is retried per `attempts / backoff`; after the final attempt it lands in the **failed** set (dead-letter). The processor sets the DB status to `error` in an `@OnWorkerEvent('failed')` handler (or after catching the terminal failure).
- The worker container registers **only** `BullModule.forRoot` + `registerQueue` + the `@Processor` (no HTTP server) — see TD-04.

bullmq (TD-01)

Source: <https://docs.bullmq.io/>. The engine underneath `@nestjs/bullmq` provides `Queue`, `Worker`, `Job` types and the retry/backoff/dead-letter semantics.

- **Retries & backoff:** per-job `attempts` + `backoff: { type: 'exponential' | 'fixed', delay }`. Exponential backoff spaces retries out (5s, 10s, 20s...).
- **Dead-letter:** BullMQ has no separate DLQ — jobs that exhaust `attempts` remain in the **failed** set (`removeOnFail: false`), which is inspectable/retryable. This is the “dead-letter” mechanism referenced in TD-01/TD-08.
- **Idempotency:** the processor must be safe to re-run (a retry re-executes `ffprobe/ffmpeg` + overwrites the thumbnail object at a deterministic `thumbnailKey`).

- **Connection:** BullMQ requires `maxRetriesPerRequest: null` on the ioredis connection for blocking commands; `@nestjs/bullmq` sets sane defaults, but when passing a custom ioredis instance, set it explicitly.

ioredis (TD-01)

Source: <https://github.com/redis/ioredis> . Redis client used by BullMQ for the connection to the `redis:7-alpine` Compose service.

```
// connection object consumed by BullModule.forRootAsync
const connection = {
  host: process.env.REDIS_HOST ?? 'redis', // Compose service name – never localhost
  port: Number(process.env.REDIS_PORT ?? 6379),
};
```

- The API producer and the worker consumer share the **same** Redis (`redis` service) so jobs enqueued by the API are seen by the worker.
- Health: the Compose `redis` service exposes `redis-cli ping` for a healthcheck; `nestjs-api` and `nestjs-worker` depends_on it with `condition: service_healthy` .

@aws-sdk/client-s3 (TD-02, TD-03, TD-07)

Source: <https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/client/s3/> . S3-compatible client pointed at MinIO. Same code path targets real S3 in prod (change endpoint/credentials only).

Client configuration for MinIO

```
import { S3Client } from '@aws-sdk/client-s3';

const s3 = new S3Client({
  endpoint: cfg.endpoint, // e.g. http://minio:9000 (Compose service name)
  region: cfg.region, // e.g. 'us-east-1' (required even for MinIO)
  credentials: { accessKeyId: cfg.accessKey, secretAccessKey: cfg.secretKey },
  forcePathStyle: true, // REQUIRED for MinIO (path-style, not virtual-host)
});
```

`forcePathStyle: true` + explicit `region / endpoint` are the MinIO compatibility settings called out in TD-02.

Multipart upload commands (TD-03)

```
import {
  CreateMultipartUploadCommand, // POST /videos → returns UploadId
  UploadPartCommand,           // presigned per part (client PUTs directly)
  CompleteMultipartUploadCommand, // complete-upload → { Parts: [{ ETag, PartNumber }] }
  AbortMultipartUploadCommand,   // orphan cleanup / cancel
} from '@aws-sdk/client-s3';
```

Flow: `CreateMultipartUpload` (API) → per-part presigned `UploadPartCommand` URL (client PUTs each chunk directly to MinIO, collects `ETag`s) → `CompleteMultipartUpload` with the ordered `{ ETag, PartNumber }` list (API). Bytes never traverse the API.

GET / streaming (TD-07)

```
import { GetObjectCommand, DeleteObjectCommand, HeadObjectCommand } from '@aws-sdk/client-s3';
```

MinIO/S3 honor `Range` on `GetObject` natively → 206 Partial Content + `Accept-Ranges: bytes`, which is what powers HTML5 `<video>` seeking without a full download.

@aws-sdk/s3-request-presigner (TD-03, TD-07)

Source: <https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/Package/-aws-sdk-s3-request-presigner/>. Generates time-limited presigned URLs.

```
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

// upload part (PUT) – TD-03
const partUrl = await getSignedUrl(
  s3,
  new UploadPartCommand({ Bucket, Key, UploadId, PartNumber }),
  { expiresIn: cfg.presignTtlSeconds }, // e.g. 3600
);

// streaming (GET, Range handled by storage) – TD-07
const streamUrl = await getSignedUrl(
  s3, new GetObjectCommand({ Bucket, Key }), { expiresIn: cfg.streamTtlSeconds },
);

// download (GET with attachment disposition) – TD-07
const downloadUrl = await getSignedUrl(
  s3,
  new GetObjectCommand({
    Bucket, Key,
    ResponseContentDisposition: `attachment; filename="${filename}"`,
  }),
  { expiresIn: cfg.downloadTtlSeconds },
);
```

- Presigned URLs embed the signature in the query string; the browser calls MinIO/S3 directly (no API in the byte path).

- `ResponseContentDisposition` overrides the response header so the same object serves both inline streaming and attachment download.
- TTLs are explicit config values (upload / stream / download), per TD-03/TD-07 policy.

nanoid (TD-06)

Source: <https://github.com/ai/nanoid> . Generates the short, URL-safe `public_id` for the public video URL. **Pin** `^3.x` — the 3.x line is CommonJS; 4+/5+/6+ are ESM-only and break the NestJS CommonJS build.

```
import { nanoid } from 'nanoid';           // 3.x = CommonJS require-compatible
const publicId = nanoid(12);               // 12 chars, URL-safe alphabet [A-Za-z0-9_-]
```

- Default length is 21 chars (~126 bits entropy); TD-06 allows a safe shorter length (~12) for cleaner URLs. At 12 chars collision risk is negligible for this scale; the `public_id` column has a **unique index**, so a (astronomically rare) collision surfaces as a `23505` and is retried on insert — same pattern as the channel-nickname retry in `ChannelsService` .
- The internal PK stays a PostgreSQL `uuid` (`@PrimaryGeneratedColumn('uuid')`), consistent with `User / Channel` . `public_id` is a separate indexed `varchar` used only in URLs.